JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util

Class WeakHashMap<K,V>

java.lang.Object
    java.util.AbstractMap<K,V>
        java.util.WeakHashMap<K,V>

Type Parameters:

`K - the type of keys maintained by this map`

`V - the type of mapped values`

All Implemented Interfaces:

`Map<K,V>`

```
public class WeakHashMap<K,V>
extends AbstractMap<K,V>
implements Map<K,V>
```

Hash table based implementation of the `Map` interface, with *weak keys*. An entry in a `WeakHashMap` will automatically be removed when its key is no longer in ordinary use. More precisely, the presence of a mapping for a given key will not prevent the key from being discarded by the garbage collector, that is, made finalizable, finalized, and then reclaimed. When a key has been discarded its entry is effectively removed from the map, so this class behaves somewhat differently from other `Map` implementations.

Both null values and the null key are supported. This class has performance characteristics similar to those of the `HashMap` class, and has the same efficiency parameters of *initial capacity* and *load factor*.

Like most collection classes, this class is not synchronized. A synchronized `WeakHashMap` may be constructed using the `Collections.synchronizedMap` method.

This class is intended primarily for use with key objects whose `equals` methods test for object identity using the == operator. Once such a key is discarded it can never be recreated, so it is impossible to do a lookup of that key in a `WeakHashMap` at some later time and be surprised that its entry has been removed. This class will work perfectly well with key objects whose `equals` methods are not based upon object identity, such as `String` instances. With such recreatable key objects, however, the automatic removal of `WeakHashMap` entries whose keys have been discarded may prove to be confusing.

The behavior of the `WeakHashMap` class depends in part upon the actions of the garbage collector, so several familiar (though not required) `Map` invariants do not hold for this class. Because the garbage collector may discard keys at any time, a `WeakHashMap` may behave as though an unknown thread is silently removing entries. In particular, even if you synchronize on a `WeakHashMap` instance and invoke none of its mutator methods, it is possible for the `size` method to return smaller values over time, for the `isEmpty` method to return `false` and then `true`, for the `containsKey` method to return `true` and later `false` for a given key, for the `get` method to return a value for a given key but later return `null`, for the `put` method to return `null` and the `remove` method to return `false` for a key that previously appeared to be in the map, and for successive examinations of the key set, the value collection, and the entry set to yield successively smaller numbers of elements.

Each key object in a `WeakHashMap` is stored indirectly as the referent of a weak reference. Therefore a key will automatically be removed only after the weak references to it, both inside and outside of the map, have been cleared by the garbage collector.

Implementation note: The value objects in a `WeakHashMap` are held by ordinary strong references. Thus care should be taken to ensure that value objects do not strongly refer to their own keys, either directly or indirectly, since that will prevent the keys from being discarded. Note that a value object may refer indirectly to its key via the `WeakHashMap` itself; that is, a value object may strongly refer to some other key object whose associated value object, in turn, strongly refers to the key of the first value object. If the values in the map do not rely on the map holding strong references to them, one way to deal with this is to wrap values themselves within `WeakReferences` before inserting, as in: `m.put(key, new WeakReference(value))`, and then unwrapping upon each `get`.

The iterators returned by the `iterator` method of the collections returned by all of this class's "collection view methods" are *fail-fast*: if the map is structurally modified at any time after the iterator is created, in any way except through the iterator's own `remove` method, the iterator will throw a `ConcurrentModificationException`. Thus, in the face of concurrent modification, the iterator fails quickly and cleanly, rather than risking arbitrary, non-deterministic behavior at an undetermined time in the future.

Note that the fail-fast behavior of an iterator cannot be guaranteed as it is, generally speaking, impossible to make any hard guarantees in the presence of unsynchronized concurrent modification. Fail-fast iterators throw `ConcurrentModificationException` on a best-effort basis. Therefore, it would be

wrong to write a program that depended on this exception for its correctness: *the fail-fast behavior of iterators should be used only to detect bugs.*

This class is a member of the Java Collections Framework.

Since:

1.2

See Also:

`HashMap`, `WeakReference`

Nested Class Summary

Nested classes/interfaces inherited from class java.util.AbstractMap

`AbstractMap.SimpleEntry<K,V>`,
`AbstractMap.SimpleImmutableEntry<K,V>`

Constructor Summary

Constructors

Constructor	Description
`WeakHashMap()`	Constructs a new, empty `WeakHashMap` with the default initial capacity (16) and load factor (0.75).
`WeakHashMap(int initialCapacity)`	Constructs a new, empty `WeakHashMap` with the given initial capacity and the default load factor (0.75).
`WeakHashMap(int initialCapacity, float loadFactor)`	Constructs a new, empty `WeakHashMap` with the given

	initial capacity and the given load factor.
`WeakHashMap(Map<? extends K,? extends V> m)`	Constructs a new `WeakHashMap` with the same mappings as the specified map.

Method Summary

Concrete Methods

Modifier and Type	Method	Description
`void`	`clear()`	Removes all of the mappings from this map.
`boolean`	`containsKey(Object key)`	Returns `true` if this map contains a mapping for the specified key.
`boolean`	`containsValue(Object value)`	Returns `true` if this map maps one or more keys to the specified value.
`Set<Map.Entry<K,V>>`	`entrySet()`	Returns a `Set` view of the mappings contained in this map.
`void`	`forEach(BiConsumer<? super K,? super V> action)`	Performs the given action for each entry in this map until all entries have been processed or the action throws an exception.

V	get(Object key)	Returns the value to which the specified key is mapped, or null if this map contains no mapping for the key.
boolean	isEmpty()	Returns true if this map contains no key-value mappings.
Set<K>	keySet()	Returns a Set view of the keys contained in this map.
V	put(K key, V value)	Associates the specified value with the specified key in this map.
void	putAll(Map<? extends K,? extends V> m)	Copies all of the mappings from the specified map to this map.
V	remove(Object key)	Removes the mapping for a key from this weak hash map if it is present.
void	replaceAll(BiFunction<? super K,? super V,? extends V> function)	Replaces each entry's value with the result of invoking the given function on that entry until all entries have been processed or the function throws an exception.
int	size()	Returns the number of key-value mappings in this map.

`Collection<V>`	`values()`	Returns a `Collection` view of the values contained in this map.

Methods inherited from class java.util.AbstractMap

`clone`, `equals`, `hashCode`, `toString`

Methods inherited from class java.lang.Object

`finalize`, `getClass`, `notify`, `notifyAll`, `wait`, `wait`, `wait`

Methods inherited from interface java.util.Map

`compute`, `computeIfAbsent`, `computeIfPresent`, `equals`, `getOrDefault`, `hashCode`, `merge`, `putIfAbsent`, `remove`, `replace`, `replace`

Constructor Detail

WeakHashMap

```
public WeakHashMap(int initialCapacity,
                   float loadFactor)
```

Constructs a new, empty `WeakHashMap` with the given initial capacity and the given load factor.

Parameters:

`initialCapacity - The initial capacity of the WeakHashMap`

`loadFactor - The load factor of the WeakHashMap`

Throws:

`IllegalArgumentException - if the initial capacity is negative, or if the load factor is nonpositive.`

WeakHashMap

```
public WeakHashMap(int initialCapacity)
```

Constructs a new, empty `WeakHashMap` with the given initial capacity and the default load factor (0.75).

Parameters:

initialCapacity - The initial capacity of the WeakHashMap

Throws:

IllegalArgumentException - if the initial capacity is negative

WeakHashMap

```
public WeakHashMap()
```

Constructs a new, empty WeakHashMap with the default initial capacity (16) and load factor (0.75).

WeakHashMap

```
public WeakHashMap(Map<? extends K,? extends V> m)
```

Constructs a new WeakHashMap with the same mappings as the specified map. The WeakHashMap is created with the default load factor (0.75) and an initial capacity sufficient to hold the mappings in the specified map.

Parameters:

m - the map whose mappings are to be placed in this map

Throws:

NullPointerException - if the specified map is null

Since:

1.3

Method Detail

size

```
public int size()
```

Returns the number of key-value mappings in this map. This result is a snapshot, and may not reflect unprocessed entries that will be removed before next attempted access because they are no longer referenced.

Specified by:

`size` in interface `Map<K,V>`

Overrides:

`size` in class `AbstractMap<K,V>`

Returns:

the number of key-value mappings in this map

isEmpty

```
public boolean isEmpty()
```

Returns `true` if this map contains no key-value mappings. This result is a snapshot, and may not reflect unprocessed entries that will be removed before next attempted access because they are no longer referenced.

Specified by:

`isEmpty` in interface `Map<K,V>`

Overrides:

`isEmpty` in class `AbstractMap<K,V>`

Returns:

true if this map contains no key-value mappings

get

```
public V get(Object key)
```

Returns the value to which the specified key is mapped, or `null` if this map contains no mapping for the key.

More formally, if this map contains a mapping from a key `k` to a value `v` such that `(key==null ? k==null : key.equals(k))`, then this method returns `v`; otherwise it returns `null`. (There can be at most one such mapping.)

A return value of `null` does not *necessarily* indicate that the map contains no mapping

for the key; it's also possible that the map explicitly maps the key to `null`. The `containsKey` operation may be used to distinguish these two cases.

Specified by:

`get` in interface `Map<K,V>`

Overrides:

`get` in class `AbstractMap<K,V>`

Parameters:

`key - the key whose associated value is to be returned`

Returns:

`the value to which the specified key is mapped, or null if this map contains no mapping for the key`

See Also:

`put(Object, Object)`

containsKey

```
public boolean containsKey(Object key)
```

Returns `true` if this map contains a mapping for the specified key.

Specified by:

`containsKey` in interface `Map<K,V>`

Overrides:

`containsKey` in class `AbstractMap<K,V>`

Parameters:

`key - The key whose presence in this map is to be tested`

Returns:

`true if there is a mapping for key; false otherwise`

put

```
public V put(K key,
             V value)
```

Associates the specified value with the specified key in this map. If the map previously contained a mapping for this key, the old value is replaced.

Specified by:

`put` in interface `Map<K,V>`

Overrides:

`put` in class `AbstractMap<K,V>`

Parameters:

`key - key with which the specified value is to be associated.`

`value - value to be associated with the specified key.`

Returns:

`the previous value associated with key, or null if there was no mapping for key. (A null return can also indicate that the map previously associated null with key.)`

putAll

```
public void putAll(Map<? extends K,? extends V> m)
```

Copies all of the mappings from the specified map to this map. These mappings will replace any mappings that this map had for any of the keys currently in the specified map.

Specified by:

`putAll in interface Map<K,V>`

Overrides:

`putAll in class AbstractMap<K,V>`

Parameters:

`m - mappings to be stored in this map.`

Throws:

`NullPointerException - if the specified map is null.`

remove

```
public V remove(Object key)
```

Removes the mapping for a key from this weak hash map if it is present. More formally, if this map contains a mapping from key `k` to value `v` such that `(key==null ? k==null : key.equals(k))`, that mapping is removed. (The map can contain at most one such mapping.)

Returns the value to which this map previously associated the key, or `null` if the map contained no mapping for the key. A return value of `null` does not *necessarily* indicate that the map contained no mapping for the key; it's also possible that the map explicitly mapped the key to `null`.

The map will not contain a mapping for the specified key once the call returns.

Specified by:

`remove in interface Map<K,V>`

Overrides:

`remove` in class `AbstractMap<K,V>`

Parameters:

`key - key whose mapping is to be removed from the map`

Returns:

`the previous value associated with key, or null if there was no mapping for key`

clear

```
public void clear()
```

Removes all of the mappings from this map. The map will be empty after this call returns.

Specified by:

`clear` in interface `Map<K,V>`

Overrides:

`clear` in class `AbstractMap<K,V>`

containsValue

```
public boolean containsValue(Object value)
```

Returns `true` if this map maps one or more keys to the specified value.

Specified by:

`containsValue` in interface `Map<K,V>`

Overrides:

`containsValue` in class `AbstractMap<K,V>`

Parameters:

`value - value whose presence in this map is to be tested`

Returns:

`true if this map maps one or more keys to the specified value`

keySet

```
public Set<K> keySet()
```

Returns a `Set` view of the keys contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through

the iterator's own `remove` operation), the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Set.remove`, `removeAll`, `retainAll`, and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

`keySet` in interface `Map<K,V>`

Overrides:

`keySet` in class `AbstractMap<K,V>`

Returns:

a set view of the keys contained in this map

values

```
public Collection<V> values()
```

Returns a `Collection` view of the values contained in this map. The collection is backed by the map, so changes to the map are reflected in the collection, and vice-versa. If the map is modified while an iteration over the collection is in progress (except through the iterator's own `remove` operation), the results of the iteration are undefined. The collection supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`, `Collection.remove`, `removeAll`, `retainAll` and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

`values` in interface `Map<K,V>`

Overrides:

`values` in class `AbstractMap<K,V>`

Returns:

a collection view of the values contained in this map

entrySet

```
public Set<Map.Entry<K,V>> entrySet()
```

Returns a `Set` view of the mappings contained in this map. The set is backed by the map, so changes to the map are reflected in the set, and vice-versa. If the map is modified while an iteration over the set is in progress (except through the iterator's own `remove` operation, or through the `setValue` operation on a map entry returned by the iterator) the results of the iteration are undefined. The set supports element removal, which removes the corresponding mapping from the map, via the `Iterator.remove`,

`Set.remove`, `removeAll`, `retainAll` and `clear` operations. It does not support the `add` or `addAll` operations.

Specified by:

`entrySet` in interface `Map<K,V>`

Specified by:

`entrySet` in class `AbstractMap<K,V>`

Returns:

a set view of the mappings contained in this map

forEach

```
public void forEach(BiConsumer<? super K,? super V> action)
```

Description copied from interface: `Map`

Performs the given action for each entry in this map until all entries have been processed or the action throws an exception. Unless otherwise specified by the implementing class, actions are performed in the order of entry set iteration (if an iteration order is specified.) Exceptions thrown by the action are relayed to the caller.

Specified by:

`forEach` in interface `Map<K,V>`

Parameters:

action - The action to be performed for each entry

replaceAll

```
public void replaceAll(BiFunction<? super K,? super V,? extends V> function)
```

Description copied from interface: `Map`

Replaces each entry's value with the result of invoking the given function on that entry until all entries have been processed or the function throws an exception. Exceptions thrown by the function are relayed to the caller.

Specified by:

`replaceAll` in interface `Map<K,V>`

Parameters:

C
```
function - the function to apply to each entry
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.